

COMPE496

C Programming Language

Concepts and Introduction

Introduction to Algorithms and Programming
Spring 2026



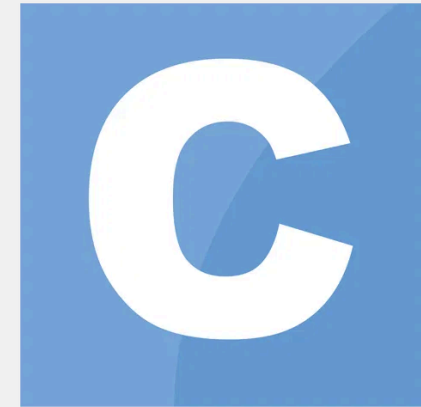
Introduction to C

Insight: C is the lingua franca of programming, bridging the gap between human intent and machine execution.

ORIGIN STORY

Created by **Dennis Ritchie** at Bell Labs in 1972 to re-write the **UNIX** operating system.

- 1960s** ALGOL 60 introduces structured programming concepts.
- 1967** BCPL (Basic Combined Programming Language) developed by Martin Richards.
- 1970** B Language created by Ken Thompson (simplified BCPL).
- 1972** **C Language** developed by adding data types to B.



Key Characteristics

- **Procedural:** Step-by-step execution.
- **Low-Level:** Direct memory access.
- **Portable:** Write once, compile anywhere.
- **Efficient:** Minimal runtime overhead.

C Language Characteristics

Insight: C provides a thin abstraction layer over hardware, offering high performance at the cost of manual management.

COMPILATION & TYPES

- **Compiled:** Source code is translated directly to machine code (binary) for the target architecture.
- **Static Typing:** Variable types are checked at compile-time, preventing type errors before execution.
- **Efficiency:** Minimal runtime overhead.

```
$ gcc main.c -o main
$ ./main
// Direct execution
```

MEMORY CONTROL

- **Manual Management:** No garbage collector. You allocate and free memory explicitly.
- **Pointers:** Direct manipulation of memory addresses.
- **Stack vs Heap:** Explicit control over variable lifetime and scope.

```
int *ptr = malloc(10);
...
free(ptr); // Required!
```

SYSTEM LEVEL

- **Small Standard Library:** Provides only essential functions (I/O, string, math).
- **Hardware Access:** Bitwise operations allow direct hardware control (drivers).
- **Portability:** Standardized (ANSI C) to run on any platform with a compiler.

```
// Bitwise control
PORTA |= (1 << 5);
// Set bit 5 high
```

Basic Syntax and Structure

Insight: Every C program starts execution at the main function, following a strict procedural structure.

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello, World!\n");
5     return 0;
6 }
```

PREPROCESSOR DIRECTIVE

Instructions for the compiler before compilation. `#include` imports the Standard I/O library for printing.

MAIN FUNCTION

The entry point of the program. Execution always begins here. `int` indicates it returns an integer status code.

STATEMENTS

The actual instructions. `printf` is a library function to output text. Statements must end with a semicolon `;`.

RETURN STATEMENT

Terminates the function. Returning `0` conventionally signals to the OS that the program finished successfully.

Data Types and Variables

Insight: C data types map directly to hardware memory. Understanding their size and range is critical for efficient programming.

TYPE	FORMAT	SIZE (TYPICAL)	USAGE
char	%c	1 byte	Characters, small integers
int	%d	4 bytes	Whole numbers
float	%f	4 bytes	Single-precision decimal
double	%lf	8 bytes	Double-precision decimal
void	-	0 bytes	No value / Generic pointer

```
DECLARATION & INITIALIZATION

int age = 25;
float price = 19.99f;
char grade = 'A';
double pi = 3.14159265;

// Check size on platform
printf("Size of int: %zu\n", sizeof(int));
```

```
FIXED-WIDTH TYPES (BEST PRACTICE)

#include <stdint.h>

// Guaranteed sizes across platforms
int32_t exact_int = 100000;
uint8_t byte_val = 255;
int64_t big_num = 9000000000LL;
```

Operators and Expressions

Insight: Operators are the verbs of C, defining how data is manipulated. Understanding precedence and types is key to correct logic.

ARITHMETIC

+ - * / %

Standard math operations. Note that integer division truncates decimals.

```
int a = 10 / 3; // Result: 3 (Truncated)
int b = 10 % 3; // Result: 1 (Remainder)
float c = 10.0 / 3; // Result: 3.333...
```

RELATIONAL

= != > < ≥ ≤

Comparisons return integer values: 1 for true, 0 for false.

```
if (x == 10) { ... } // Equality check
if (x = 10) { ... } // ERROR: Assignment!
int res = (5 > 3); // res is 1
```

LOGICAL

&& || !

Combines boolean conditions. Uses **short-circuit evaluation**.

```
// If ptr is NULL, *ptr is NOT accessed
if (ptr != NULL && *ptr > 0) {
    process(*ptr);
}
```

ASSIGNMENT & UNARY

= += ++ --

Modifies variables. Pre-increment (++i) vs Post-increment (i++) matters.

```
x += 5; // x = x + 5
y = x++; // y gets x, then x increments
z = ++x; // x increments, then z gets x
```

Control Structures

Insight: Control flow dictates the execution path, allowing programs to make decisions and repeat tasks efficiently.

DECISION MAKING

Conditional Branching (if-else)

```
if (score ≥ 90) { grade = 'A'; } else if (score ≥ 80) { grade = 'B'; } else { grade = 'F'; }
```

Multi-way Branching (switch)

```
switch (day) { case 1: printf("Monday"); break; case 2: printf("Tuesday"); break; default: printf("Unknown"); }
```

ITERATION (LOOPS)

Determinate Loop (for)

```
// Loop 10 times for (int i = 0; i < 10; i++) { printf("%d ", i); }
```

Indeterminate Loop (while)

```
int count = 0; while (count < 5) { printf("%d ", count); count++; }
```

Post-Test Loop (do-while)

```
do { // Executes at least once scanf("%d", &num); } while (num ≠ 0);
```

Arrays

Insight: Arrays are fixed-size, contiguous blocks of memory that allow constant-time access to elements via indices.

Contiguous Memory

Elements are stored side-by-side in memory. The array name acts as a pointer to the first element.

```
int arr[5] = {10, 20, 30, 40, 50};
```

10	20	30	40	50
0	1	2	3	4

Key Properties

- **0-indexed:** First element is at index 0.
- **Fixed Size:** Cannot change after declaration.
- **Homogeneous:** All elements same type.

1D DECLARATION & ACCESS

```
// Declaration & Initialization int numbers[5] = {10, 20, 30, 40, 50}; //  
Accessing elements (Read/Write) numbers[0] = 15; // Update first element  
int x = numbers[2]; // Read third element (30) // Calculating size int len =  
sizeof(numbers) / sizeof(numbers[0]);
```

MULTIDIMENSIONAL ARRAYS

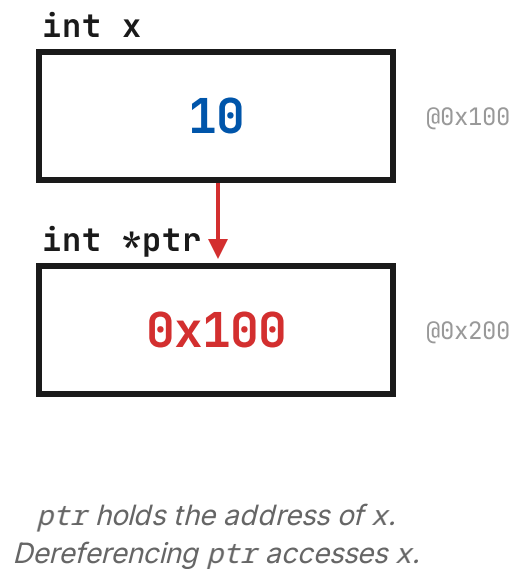
```
// 2D Array (Matrix) int matrix[3][3] = { {1, 0, 0}, {0, 1, 0}, {0, 0, 1} };  
int val = matrix[1][1]; // Access row 1, col 1
```

⚠ SAFETY WARNING: NO BOUNDS CHECKING

C does not check if an index is valid. Accessing `arr[5]` in a size-5 array leads to **Undefined Behavior** (crashes, data corruption, security exploits).

Pointers Fundamentals

Insight: A pointer is simply a variable that stores the memory address of another variable, enabling indirect access and manipulation.



ADDRESS-OF (&)

`&x`

Returns the memory address of variable x.

DEREFERENCE (*)

`*ptr`

Accesses the value stored at the address held by ptr.

```
int x = 10; int *ptr = &x; // ptr points to x printf("Value of x: %d\n", x);  
// 10 printf("Address of x: %p\n", &x); // 0x100 printf("Value of ptr:  
%p\n", ptr); // 0x200 // Indirection (Dereferencing) *ptr = 20; // Change x  
via ptr printf("New x: %d\n", x); // 20
```

Pointers and Arrays

Insight: In C, array indexing is essentially syntactic sugar for pointer arithmetic. The array name acts as a constant pointer to the first element.

The Relationship

An array name (without brackets) evaluates to the memory address of its first element (index 0).

Pointer Arithmetic

Adding 1 to a pointer increases the address by `sizeof(type)` bytes, effectively moving to the next element.

THE EQUIVALENCE RULE

$$\text{arr}[i] = \text{*(arr + i)}$$

"Go to address (arr + i) and get the value"

ARRAY NAME AS POINTER

```
int arr[5] = {10, 20, 30, 40, 50}; int *p = arr; // Points to arr[0]
printf("%d\n", *p); // Prints 10 printf("%d\n", *(p+1)); // Prints 20 (Next
int) printf("%d\n", *(p+2)); // Prints 30
```

ITERATING WITH POINTERS

```
// Efficient traversal (common in systems code) for (int *ptr = arr; ptr <
arr + 5; ptr++) { printf("%d ", *ptr); } // ptr++ moves to the next integer
automatically
```

SYNTACTIC CONSEQUENCE

```
// Since a + b = b + a // arr[2] = *(arr + 2) = *(2 + arr) = 2[arr] int
val = 2[arr]; // Valid C code (but don't do this!)
```

Functions

Insight: Functions modularize code. In C, understanding the distinction between passing by value (copy) and passing by reference (pointer) is fundamental.

STRUCTURE

1. Declaration (Prototype)

Tells compiler about function name and types before it's defined.

2. Definition

The actual implementation of the logic.

3. Call

Executing the function.

```
// 1. Prototype int add(int a, int b); int main() {  
// 3. Call int sum = add(5, 3); return 0; } // 2.  
Definition int add(int a, int b) { return a + b; }
```

PARAMETER PASSING

PASS BY VALUE

Sends a **copy** of the data. Original variable remains unchanged.

```
void inc(int x) { x++; // Local  
copy } int num = 5; inc(num); //  
num is still 5
```

PASS BY REFERENCE

Sends the **address** (pointer). Function modifies original memory.

```
void incRef(int *x) { (*x)++; //  
Modifies original } int num = 5;  
incRef(&num); // num is now 6
```

Structures (struct)

Insight: Structures allow grouping of related variables of different types under a single name, forming the basis of complex data modeling.

Data Grouping

Unlike arrays (homogeneous), structures are **heterogeneous** containers. They map logical entities (like a Student or a Point) to memory.

Access Operators

• **Dot Operator**
Access member of an object instance.

→ **Arrow Operator**
Access member via a pointer.

DEFINITION & INSTANCE ACCESS

```
struct Student { char name[50]; int id; float gpa; }; // Initialization
struct Student s1 = {"John Doe", 12345, 3.75}; // Access with Dot (.)
printf("Name: %s\n", s1.name); s1.gpa = 3.80;
```

POINTER ACCESS

```
struct Student *ptr = &s1; // Access with Arrow (→) // Equivalent to
(*ptr).id printf("ID: %d\n", ptr→id); // Modifying via pointer ptr→gpa =
4.0;
```

Dynamic Memory Allocation

Insight: Dynamic allocation allows programs to request memory from the heap at runtime, essential for data structures whose size isn't known beforehand.

MALLOC

```
void* malloc(size_t size)
```

Allocates a block of uninitialized memory. Contains garbage values initially.

```
// Allocate array of 5 ints
int *arr = (int*)malloc(5 * sizeof(int));
if (arr == NULL) { /* Handle Error */ }
```

CALLOC

```
void* calloc(size_t n, size_t size)
```

Allocates memory for an array of elements and **initializes them to zero**.

```
// Allocate 5 ints, all set to 0
int *arr = (int*)calloc(5, sizeof(int));
```

REALLOC

```
void* realloc(void* ptr, size_t size)
```

Resizes a previously allocated block. Preserves existing data up to the new size.

```
// Resize to hold 10 ints
arr = (int*)realloc(arr, 10 * sizeof(int));
```

FREE

```
void free(void* ptr)
```

Releases allocated memory back to the system. **Crucial** to prevent memory leaks.

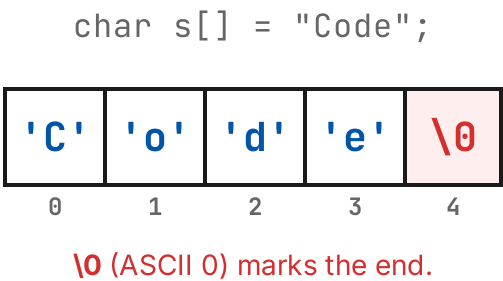
```
free(arr);
arr = NULL; // Prevent dangling pointer
```

Strings in C

Insight: In C, a string is not a primitive type but a sequence of characters terminated by a null byte (`\0`).

Character Arrays

Strings are stored as arrays of `char`. The end of the string is marked by a special sentinel value: the Null Terminator.



Key Implications

- Size is **Length + 1** (for null).
- Missing `\0` causes buffer overruns.
- `strlen()` counts chars *before* `\0`.

Standard Operations (<string.h>)

OPERATION	UNSAFE (AVOID)	SAFE (PREFERRED)
Copy	<code>strcpy()</code>	<code>strncpy()</code>
Concatenate	<code>strcat()</code>	<code>strncat()</code>
Input	<code>gets()</code>	<code>fgets()</code>
Length	<code>strlen()</code>	<code>strnlen()</code>

```
// Safe String Handling Example
char buffer[10]; // 1. Safe Input (prevents overflow)
fgets(buffer, sizeof(buffer), stdin); // 2. Safe Copy
char dest[10];
strncpy(dest, buffer, sizeof(dest) - 1);
dest[sizeof(dest) - 1] = '\0'; // Ensure termination
// 3. Compare
if (strcmp(dest, "Hello") == 0) {
    printf("Match!\n");
}
```

File I/O

Insight: File operations in C rely on file pointers and streams. Robust error handling is essential for reliable I/O.

WRITING TO FILES

- Open with "w" (write) or "a" (append).
- Always check if the file pointer is NULL.
- Use fprintf for formatted text output.

```
FILE *fp = fopen("output.txt", "w"); // 1. Check for error if
(fp == NULL) { perror("Error opening file"); return 1; } // 2.
Write data fprintf(fp, "Score: %d\n", 100); fputs("Game
Over\n", fp); // 3. Close stream fclose(fp);
```

READING FROM FILES

- Open with "r" (read).
- Use fgets to read line-by-line safely.
- Loop until NULL (End of File).

```
FILE *fp = fopen("input.txt", "r"); char buffer[100]; if (fp ==
NULL) return 1; // Read line by line until EOF while
(fgets(buffer, sizeof(buffer), fp) != NULL) { // Process line
printf("Read: %s", buffer); } fclose(fp);
```

C Best Practices

Insight: C places the burden of safety on the programmer. Adopting strict discipline in memory and buffer management is essential.

Explicit Control

```
x=10  
if x>5:  
    print('Negative')  
if x<5:  
    print('Positive')
```

Negative

```
x=10;  
if x>5:  
    disp('Negative')  
if x<5:  
    disp('Positive')
```

end
end
Negative

C is imperative. You define every step of the state change. This power requires you to manually ensure safety at every step.

MEMORY HYGIENE

- Check malloc return values.
- Pair every allocation with a free.

```
ptr = malloc(size);  
if (ptr == NULL) return;  
...  
free(ptr);
```

BUFFER SECURITY

- Never use gets().
- Use bounds-checking functions.

```
gets(buf);  
fgets(buf, size, stdin);  
// Prevents overflow
```

POINTER SAFETY

- Initialize to NULL.
- Nullify after freeing (Dangling pointer).

```
free(ptr);  
ptr = NULL;  
// Safe from double-free
```

TYPE CLARITY

- Use <stdint.h> for fixed sizes.
- Use const for immutability.

```
long val;  
int32_t val;  
// Portable size
```